ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Bachelor's Thesis Proposal

Secure Object-Relational Mappers

Supervisor: Dr. Srđan Krstić, Hoang Nguyen
Professor: Prof. David Basin

1 Prerequisites

Familiarity with databases, object-relational mappers, SQL, and model-driven development. [1]

2 Motivation

Enforcing security policies, in particular, fine-grained access control (FGAC), in today's data-centric applications is both critical and challenging. It is critical because of the high economic (fines) and social (bad reputation) consequences of failing to enforce them over privacy-sensitive data. As enforcing FGAC policies is not yet well-supported by the current technologies, it is often implemented in an ad-hoc manner, which is error-prone and time consuming.

Nguyen and Clavel [7] propose FGAC model and its characterization for SQL queries. Based on this characterization, they have implemented an enforcement mechanism as a software component called `SQLSI` [8, 3]. `SQLSI` rewrites an SQL `SELECT` query into an SQL stored procedure by *injecting* authorization checks into the stored procedure, based on the FGAC model. Intuitively, whenever a query is to be executed, instead of executing it directly without any security mechanism in place, the database system executes the corresponding stored-procedure that is generated by `SQLSI` and secure according to the FGAC model.

In practice, typical three-tier system architectures use an Object Relational Mapper (ORM) that bridges the gap between the system's application and database layers. Instead of writing SQL queries, developers execute actions (e.g., reading an attribute) on data persistent classes, which ORM translates into the equivalent SQL queries. This hides and simplifies developers' interaction with the database.

3 Objective

In this study, we would like to extend a selected Object Relational Mapper to support secured queries, with respect to the proposed characterization [7]. Furthermore, we would like to generalize the extension to other ORMs, optimize and extend `SQLSI` itself. The main goal is to support developers with a powerful security framework in a usable way without restricting their freedom to modify and evolve the applications. To evaluate whether this goal is met we plan to showcase applicability of the extended ORM over a set of realistic case studies.

[1] E.g., see the course on Security Engineering: `https://infsec.ethz.ch/education/as2022/seceng.html`

4 Tasks

1. Study a selected ORM implementation (e.g., `LINQ` [1, 4], or `SQLAlchemy` [5, 6])
2. Understand the proposed characterization in [7], the implementation of `SQLSI` [8].
3. For a selected ORM implement a naive extension that generates a stored procedure and executes it with the appropriate context.
4. Optimize stored procedure generation not to re-generate existing procedures.
5. Analyze the code and generate statically as many stored procedures as possible.
6. (Optional) Generalize the approach to multiple different ORMs.
7. (Optional) Extend `SQLSI` with support for SQL `INSERT`, `UPDATE` and `DELETE` queries.
8. Develop case studies (e.g., `MiniTwit` [2]) using the new approach, compare it to existing approaches, and assess its usability and performance.

5 Deliverables

The following deliverables are due at the end of the project:

Final report The final report should consist of an introduction; a theoretical background section; one or more sections describing the modeling, implementation and evaluation; and a conclusion. The report may be written in English or German.

Source code Complete development including easily reproducible evaluation.

Presentation At the end of the project, a 20 minute presentation must be given in an InfSec group seminar. It should give an overview and discuss the most important highlights of the work.

References

References

[1] Language-Integrated Query (LINQ) documentation, 2018. `https://learn.microsoft.com/en-us/dotnet/api/system.linq`.

[2] MiniTwit: A tiny sample Flask app featuring a database and user accounts, November 2022. `https://github.com/Viasat/minitwit`.

[3] The SQLSI Tool. `https://github.com/MoDELSVGU/SQLSI`, 2022.

[4] LINQ implementation, March 2023. `https://github.com/dotnet/runtime/tree/main/src/libraries/System.Linq`.

[5] SQLAlchemy 2.0 documentation, March 2023. `https://docs.sqlalchemy.org/en/20/`.

[6] SQLAlchemy implementation, 2023. `https://github.com/sqlalchemy/sqlalchemy`.

[7] Hoang Nguyen and Manuel Clavel. Model-based Characterization of fine-grained Access Control Authorization for SQL Queries. *J. Object Technol.*, 19(3):3:1–13, 2020.

[8] Hoang Nguyen and Manuel Clavel. A Model-Driven Approach for Enforcing Fine-Grained Access Control for SQL Queries. *SN Comput. Sci.*, 2(5):370, 2021.